

Documentacion de Controladores

Sistema de Prestamo de Equipo Tecnologico
TechSolutions S.A.

ASP.NET MVC 5 . .NET Framework 4.8 . Entity Framework 6 . ASP.NET Identity

Generado el 07/08/2026

Introduccion

Este documento explica, uno por uno, los 7 controladores del sistema. El proyecto esta organizado por capas: los Controllers (lo que aqui se documenta) reciben la peticion HTTP y devuelven una vista, pero NO contienen la logica de negocio ni hablan directo con la base de datos; para eso usan la capa de Servicios (Servicios/), que a su vez usa la capa de Repositorios (Datos/Repositorios/) para leer y escribir en Entity Framework.

El sistema usa ASP.NET Identity para el login y maneja exactamente dos roles fijos:

- Administrador: acceso total. Puede administrar Equipos, Empleados, Prestamos (incluyendo eliminarlos), Usuarios y Roles.
- Operador: solo puede ver, registrar, editar y devolver Prestamos. No tiene acceso a Equipos, Empleados, Usuarios ni Roles, y no puede eliminar prestamos.

Las tres entidades de negocio (Equipo, Empleado, Prestamo) se relacionan asi: un Equipo puede tener muchos Prestamos, un Empleado puede tener muchos Prestamos, y la tabla Prestamos conecta a ambos mediante las llaves foraneas Equipold y Empleadold.

1. AccountController

Para que sirve

Es el controlador de autenticacion de todo el sistema, construido sobre ASP.NET Identity. Se encarga de tres cosas: mostrar el formulario de inicio de sesion y validar usuario/contrasena (Login), cerrar la sesion actual (LogOff), y crear cuentas de usuario nuevas asignandoles un rol (Register). Esta ultima accion es exclusiva del rol Administrador: en este sistema nadie se auto-registra, solo un administrador puede dar de alta a otra persona. No tiene un CRUD propio en base de datos: delega el trabajo en los 'managers' de Identity (userManager, SignInManager, RoleManager), que internamente leen y escriben en las tablasAspNetUsers, AspNetRoles y AspNetUserRoles creadas por el script SQL.

Codigo completo (AccountController.cs)

```
using System.Linq;
using System.Threading.Tasks;
using System.Web;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.ViewModels;
using Microsoft.AspNet.Identity;
using Microsoft.AspNet.Identity.Owin;
using Microsoft.Owin;
using Microsoft.Owin.Security;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    // =====
    // AccountController
    // =====
    // Controlador de autenticación del sistema, basado en ASP.NET Identity.
    // Se encarga de: iniciar sesión (Login), cerrar sesión (LogOff) y crear
    // nuevos usuarios asignándoles un rol (Register, solo para Administrador).
    // No tiene CRUD propio: usa los "managers" de Identity (userManager,
    // SignInManager, RoleManager) que trabajan sobre las tablas AspNetUsers,
    // AspNetRoles y AspNetUserRoles creadas por el script SQL.
    // =====
    public class AccountController : Controller
    {
        private ApplicationSignInManager _signInManager;
        private ApplicationUserManager _userManager;
        private ApplicationRoleManager _roleManager;

        public ApplicationSignInManager SignInManager =>
            _signInManager ?? (_signInManager = HttpContext.GetOwinContext().Get<ApplicationSignInManager>());

        public ApplicationUserManager UserManager =>
            _userManager ?? (_userManager = HttpContext.GetOwinContext().GetUserManager<ApplicationUserManager>());

        public ApplicationRoleManager RoleManager =>
            _roleManager ?? (_roleManager = HttpContext.GetOwinContext().Get<ApplicationRoleManager>());

        private IAuthenticationManager AuthenticationManager => HttpContext.GetOwinContext().Authentication;

        // GET: Account/Login
        // Muestra el formulario de inicio de sesión. [AllowAnonymous] porque
        // cualquiera (sin haber iniciado sesión) debe poder ver esta pantalla.
        [HttpGet]
        [AllowAnonymous]
        public ActionResult Login(string returnUrl)
        {

```

```

        ViewBag.ReturnUrl = returnUrl;
        return View(new LoginViewModel());
    }

    // POST: Account/Login
    // Valida usuario/contraseña contra AspNetUsers (PasswordSignInAsync) y,
    // si son correctos, crea la cookie de autenticación. Si venía de una
    // página protegida (returnUrl), regresa ahí; si no, va al Dashboard.
    [HttpPost]
    [AllowAnonymous]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> Login(LoginViewModel modelo, string returnUrl)
    {
        ViewBag.ReturnUrl = returnUrl;

        if (!ModelState.IsValid)
        {
            return View(modelo);
        }

        var resultado = await SignInManager.PasswordSignInAsync(
            modelo.Email, modelo.Password, modelo.RememberMe, shouldLockout: true);

        switch (resultado)
        {
            case SignInStatus.Success:
                if (Url.IsLocalUrl(returnUrl))
                {
                    return Redirect(returnUrl);
                }
                return RedirectToAction("Index", "Home");

            case SignInStatus.LockedOut:
                ModelState.AddModelError("", "Esta cuenta ha sido bloqueada temporalmente por múltiples intentos fallidos.");
                return View(modelo);

            default:
                ModelState.AddModelError("", "Correo o contraseña incorrectos.");
                return View(modelo);
        }
    }

    // POST: Account/LogOff
    // Cierra la sesión actual (borra la cookie de autenticación de Identity).
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult LogOff()
    {
        AuthenticationManager.SignOut(DefaultAuthenticationTypes.ApplicationCookie);
        return RedirectToAction("Login", "Account");
    }

    // GET: Account/Register
    // Formulario para crear un usuario nuevo. Solo lo puede abrir un
    // Administrador (los usuarios no se auto-registran en este sistema).
    [HttpGet]
    [Authorize(Roles = "Administrador")]
    public ActionResult Register()
    {
        ViewBag.Roles = RoleManager.Roles.Select(r => r.Name).OrderBy(r => r).ToList();
        return View(new RegisterViewModel());
    }

    // POST: Account/Register
    // Crea el usuario en AspNetUsers (CreateAsync) y le asigna el rol elegido
    // (AddToRoleAsync). Si la asignación de rol falla, se borra el usuario

```

```

    // recién creado para no dejar una cuenta "sin rol" que nunca podría
    // entrar a ningún módulo del sistema.
    [HttpPost]
    [Authorize(Roles = "Administrador")]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> Register(RegisterViewModel modelo)
    {
        ViewBag.Roles = RoleManager.Roles.Select(r => r.Name).OrderBy(r => r).ToList();

        if (!ModelState.IsValid)
        {
            return View(modelo);
        }

        var usuario = new ApplicationUser
        {
            UserName = modelo.Email,
            Email = modelo.Email,
            NombreCompleto = modelo.NombreCompleto
        };

        var resultado = await UserManager.CreateAsync(usuario, modelo.Password);

        if (resultado.Succeeded)
        {
            var resultadoRol = await UserManager.AddToRoleAsync(usuario.Id, modelo.Rol);

            if (!resultadoRol.Succeeded)
            {
                // El usuario ya se creó pero se quedó sin rol: sin rol no puede
                // entrar a ningún módulo (se vería como un "bucle" de login).
                // Se revierte la creación para no dejar cuentas huérfanas.
                await UserManager.DeleteAsync(usuario);
                foreach (var error in resultadoRol.Errors)
                {
                    ModelState.AddModelError("", "No se pudo asignar el rol: " + error);
                }
                return View(modelo);
            }

            TempData["MensajeExito"] = $"Usuario {modelo.Email} creado correctamente con el rol {modelo.Rol}.";
            return RedirectToAction("Index", "Usuarios");
        }

        foreach (var error in resultado.Errors)
        {
            ModelState.AddModelError("", error);
        }

        return View(modelo);
    }

    protected override void Dispose(bool disposing)
    {
        if (disposing)
        {
            _userManager?.Dispose();
            _signInManager?.Dispose();
            _roleManager?.Dispose();
        }
        base.Dispose(disposing);
    }
}

```

2. HomeController

Para que sirve

Muestra el Dashboard: la pantalla principal que ve cualquier usuario ya autenticado, sin importar su rol ([Authorize] sin especificar Roles). Su unica accion (Index) cuenta cuantos equipos hay en total, cuantos estan disponibles o prestados, cuantos empleados existen y cuantos prestamos estan activos o ya fueron devueltos, y manda todos esos numeros a la vista en un ViewModel (PanelInicioViewModel) para pintar las tarjetas del panel.

Codigo completo (HomeController.cs)

```
using System.Linq;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.Modelos;
using Caso.Uno.Principal.Front.views.ViewModels;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    // =====
    // HomeController
    // =====
    // Muestra el Dashboard: la primera pantalla que ve cualquier usuario ya
    // autenticado (Administrador u Operador, [Authorize] sin rol específico).
    // Solo lee estadísticas (conteos) directo de la base de datos para pintar
    // las tarjetas del panel; no tiene CRUD propio.
    // =====
    [Authorize]
    public class HomeController : Controller
    {
        private readonly ApplicationDbContext db = new ApplicationDbContext();

        // GET: Home/Index
        // Cuenta equipos (totales/disponibles/prestados), empleados y préstamos
        // (activos/devueltos) y los manda a la vista como PanelInicioViewModel.
        public ActionResult Index()
        {
            var modelo = new PanelInicioViewModel
            {
                TotalEquipos = db.Equipos.Count(),
                EquiposDisponibles = db.Equipos.Count(e => e.Estado == EstadoEquipo.Disponible),
                EquiposPrestados = db.Equipos.Count(e => e.Estado == EstadoEquipo.Prestado),
                TotalEmpleados = db.Empleados.Count(),
                PrestamosActivos = db.Prestamos.Count(p => p.Estatus == EstatusPrestamo.Prestado),
                PrestamosDevueltos = db.Prestamos.Count(p => p.Estatus == EstatusPrestamo.Devuelto)
            };

            return View(modelo);
        }

        protected override void Dispose(bool disposing)
        {
            if (disposing) db.Dispose();
            base.Dispose(disposing);
        }
    }
}
```

3. EquiposController

Para que sirve

Implementa el CRUD completo (Crear, Leer, Actualizar, Eliminar) del catalogo de Equipos tecnologicos: Nombre, Marca, Modelo, Serie y Estado. Es exclusivo del rol Administrador (el Operador no tiene este modulo en su menu ni acceso a sus URLs). Sigue la arquitectura por capas del proyecto: el controlador no habla directo con Entity Framework, sino que usa EquipoServicio (capa de logica de negocio), el cual a su vez usa EquipoRepositorio y PrestamoRepositorio (capa de acceso a datos). Gracias a esa separacion, el servicio puede aplicar una regla de negocio importante: no se puede eliminar un equipo que ya tiene prestamos registrados.

Codigo completo (EquiposController.cs)

```
using System;
using System.Data.Entity;
using System.Net;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.Datos.Repositorios;
using Caso.Uno.Principal.Front.views.Modelos;
using Caso.Uno.Principal.Front.views.Servicios;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    // =====
    // EquiposController
    // =====
    // CRUD completo del catálogo de Equipos tecnológicos (Nombre, Marca, Modelo,
    // Serie, Estado). Solo lo puede usar el Administrador: el Operador no ve
    // este módulo (por eso el rol Operador no aparece en [Authorize]).
    // No habla directo con Entity Framework: delega en EquipoServicio (capa de
    // negocio), que a su vez usa EquipoRepositorio/PrestamoRepositorio (capa de
    // datos). Esa separación es la "arquitectura por capas" del proyecto.
    // =====
    [Authorize(Roles = "Administrador")]
    public class EquiposController : Controller
    {
        private readonly ApplicationDbContext db = new ApplicationDbContext();
        private readonly EquipoServicio _servicio;

        public EquiposController()
        {
            _servicio = new EquipoServicio(new EquipoRepositorio(db), new PrestamoRepositorio(db));
        }

        // GET: Equipos/Index?buscar=texto
        // Lista todos los equipos; si viene "buscar", filtra por nombre, marca,
        // modelo, serie o estado (ver EquipoRepositorio.Buscar).
        public ActionResult Index(string buscar)
        {
            ViewBag.Buscar = buscar;
            return View(_servicio.Buscar(buscar));
        }

        // GET: Equipos/Details/5 ? muestra el detalle de un equipo.
        public ActionResult Details(int? id)
        {
            if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
            var equipo = _servicio.ObtenerPorId(id.Value);
            if (equipo == null) return HttpNotFound();
        }
    }
}
```

```

        return View(equipo);
    }

    // GET: Equipos/Create ? formulario para registrar un equipo nuevo.
    public ActionResult Create()
    {
        ViewBag.Estados = EstadoEquipo.Todos;
        return View(new Equipo());
    }

    // POST: Equipos/Create ? valida el modelo y guarda el equipo nuevo.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Create([Bind(Include = "Id,Nombre,Marca,Modelo,Serie,Estado")] Equipo equipo)
    {
        if (ModelState.IsValid)
        {
            _servicio.Registrar(equipo);
            TempData["MensajeExito"] = "Equipo registrado correctamente.";
            return RedirectToAction("Index");
        }

        ViewBag.Estados = EstadoEquipo.Todos;
        return View(equipo);
    }

    // GET: Equipos/Edit/5 ? formulario de edición con los datos actuales.
    public ActionResult Edit(int? id)
    {
        if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
        var equipo = _servicio.ObtenerPorId(id.Value);
        if (equipo == null) return HttpNotFound();
        ViewBag.Estados = EstadoEquipo.Todos;
        return View(equipo);
    }

    // POST: Equipos/Edit/5 ? guarda los cambios del equipo.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Edit([Bind(Include = "Id,Nombre,Marca,Modelo,Serie,Estado")] Equipo equipo)
    {
        if (ModelState.IsValid)
        {
            _servicio.Actualizar(equipo);
            TempData["MensajeExito"] = "Equipo actualizado correctamente.";
            return RedirectToAction("Index");
        }

        ViewBag.Estados = EstadoEquipo.Todos;
        return View(equipo);
    }

    // GET: Equipos/Delete/5 ? pantalla de confirmación antes de eliminar.
    public ActionResult Delete(int? id)
    {
        if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
        var equipo = _servicio.ObtenerPorId(id.Value);
        if (equipo == null) return HttpNotFound();
        return View(equipo);
    }

    // POST: Equipos/Delete/5 ? elimina el equipo. EquipoServicio.Eliminar
    // lanza InvalidOperationException si el equipo tiene préstamos
    // registrados, para no romper la integridad referencial con Prestamos.
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]

```



```
public ActionResult DeleteConfirmed(int id)
{
    try
    {
        _servicio.Eliminar(id);
        TempData["MensajeExito"] = "Equipo eliminado correctamente.";
    }
    catch (InvalidOperationException ex)
    {
        TempData["MensajeError"] = ex.Message;
    }

    return RedirectToAction("Index");
}

protected override void Dispose(bool disposing)
{
    if (disposing) db.Dispose();
    base.Dispose(disposing);
}
}
```

4. EmpleadosController

Para que sirve

Implementa el CRUD completo del catalogo de Empleados (Nombre, Departamento, Correo, Telefono): son las personas de la empresa a quienes se les puede prestar un equipo. Tiene exactamente la misma estructura por capas que EquiposController (Controller -> EmpleadoServicio -> EmpleadoRepositorio) y tambien es exclusivo de Administrador. El servicio impide eliminar a un empleado que ya tiene prestamos registrados, para no romper la integridad referencial de la tabla Prestamos.

Codigo completo (EmpleadosController.cs)

```
using System;
using System.Net;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.Datos.Repositorios;
using Caso.Uno.Principal.Front.views.Modelos;
using Caso.Uno.Principal.Front.views.Servicios;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    // =====
    // EmpleadosController
    // =====
    // CRUD completo del catálogo de Empleados (Nombre, Departamento, Correo,
    // Teléfono): son las personas a quienes se les puede prestar un equipo.
    // Igual que EquiposController, es exclusivo de Administrador y delega la
    // lógica de negocio en EmpleadoServicio (capa de Servicios).
    // =====
    [Authorize(Roles = "Administrador")]
    public class EmpleadosController : Controller
    {
        private readonly ApplicationDbContext db = new ApplicationDbContext();
        private readonly EmpleadoServicio _servicio;

        public EmpleadosController()
        {
            _servicio = new EmpleadoServicio(new EmpleadoRepositorio(db), new PrestamoRepositorio(db));
        }

        // GET: Empleados/Index?buscar=texto ? lista con buscador.
        public ActionResult Index(string buscar)
        {
            ViewBag.Buscar = buscar;
            return View(_servicio.Buscar(buscar));
        }

        // GET: Empleados/Details/5 ? detalle de un empleado.
        public ActionResult Details(int? id)
        {
            if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
            var empleado = _servicio.ObtenerPorId(id.Value);
            if (empleado == null) return HttpNotFound();
            return View(empleado);
        }

        // GET: Empleados/Create ? formulario de alta.
        public ActionResult Create()
        {
            return View(new Empleado());
        }
    }
}
```

```

// POST: Empleados/Create ? guarda el empleado nuevo.
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Create([Bind(Include = "Id,Nombre,Departamento,Correo,Telefono")] Empleado empleado)
{
    if (ModelState.IsValid)
    {
        _servicio.Registrar(empleado);
        TempData["MensajeExito"] = "Empleado registrado correctamente.";
        return RedirectToAction("Index");
    }

    return View(empleado);
}

// GET: Empleados/Edit/5 ? formulario de edición.
public ActionResult Edit(int? id)
{
    if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    var empleado = _servicio.ObtenerPorId(id.Value);
    if (empleado == null) return HttpNotFound();
    return View(empleado);
}

// POST: Empleados/Edit/5 ? guarda los cambios.
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Edit([Bind(Include = "Id,Nombre,Departamento,Correo,Telefono")] Empleado empleado)
{
    if (ModelState.IsValid)
    {
        _servicio.Actualizar(empleado);
        TempData["MensajeExito"] = "Empleado actualizado correctamente.";
        return RedirectToAction("Index");
    }

    return View(empleado);
}

// GET: Empleados/Delete/5 ? confirmación antes de eliminar.
public ActionResult Delete(int? id)
{
    if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    var empleado = _servicio.ObtenerPorId(id.Value);
    if (empleado == null) return HttpNotFound();
    return View(empleado);
}

// POST: Empleados/Delete/5 ? elimina, salvo que tenga préstamos
// registrados (EmpleadoServicio.Eliminar lanza la excepción en ese caso).
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public ActionResult DeleteConfirmed(int id)
{
    try
    {
        _servicio.Eliminar(id);
        TempData["MensajeExito"] = "Empleado eliminado correctamente.";
    }
    catch (InvalidOperationException ex)
    {
        TempData["MensajeError"] = ex.Message;
    }

    return RedirectToAction("Index");
}

```

```
    }  
  
    protected override void Dispose(bool disposing)  
    {  
        if (disposing) db.Dispose();  
        base.Dispose(disposing);  
    }  
}
```

5. PrestamosController

Para que sirve

Es el modulo central del sistema: relaciona un Equipo con un Empleado para registrar un prestamo (llaves foraneas Equipold y EmpleadoId en la tabla Prestamos) y controla todo el flujo hasta la devolucion. Es el unico controlador compartido por los dos roles, pero con permisos distintos: Administrador puede ver, crear, editar, devolver y ELIMINAR prestamos; Operador puede ver, crear, editar y devolver, pero no eliminar (las acciones Delete/DeleteConfirmed tienen su propio atributo [Authorize(Roles = "Administrador")] adicional al de la clase). La logica de negocio vive en PrestamoServicio: valida que el equipo no este ya prestado antes de crear un prestamo nuevo, y al registrar una devolucion libera el equipo (lo regresa a Estado = 'Disponible').

Codigo completo (PrestamosController.cs)

```
using System;
using System.Net;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.Datos.Repositorios;
using Caso.Uno.Principal.Front.views.Modelos;
using Caso.Uno.Principal.Front.views.Servicios;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    // =====
    // PrestamosController
    // =====
    // Controla el proceso de préstamo/devolución de equipo: relaciona un Equipo
    // con un Empleado (llaves foráneas EquipoId/EmpleadoId en la tabla Prestamos).
    // Es el único módulo que ven AMBOS roles, pero con permisos distintos:
    // - Administrador: puede ver, crear, editar, devolver y ELIMINAR.
    // - Operador: puede ver, crear, editar y devolver, pero NO eliminar
    // (por eso Delete/DeleteConfirmed tienen su propio
    // [Authorize(Roles = "Administrador")] adicional).
    // La lógica de negocio (que no se pueda prestar un equipo ya prestado, que
    // al devolver se libere el equipo, etc.) vive en PrestamoServicio, no aquí.
    // =====
    [Authorize(Roles = "Administrador,Operador")]
    public class PrestamosController : Controller
    {
        private readonly ApplicationDbContext db = new ApplicationDbContext();
        private readonly PrestamoServicio _servicio;
        private readonly IEquipoRepositorio _equipoRepositorio;
        private readonly IEmpleadoRepositorio _empleadoRepositorio;

        public PrestamosController()
        {
            _equipoRepositorio = new EquipoRepositorio(db);
            _empleadoRepositorio = new EmpleadoRepositorio(db);
            _servicio = new PrestamoServicio(new PrestamoRepositorio(db), _equipoRepositorio);
        }

        // GET: Prestamos/Index?buscar=texto
        // Lista los préstamos (con el Equipo y Empleado ya incluidos, ver
        // PrestamoRepositorio.BuscarConDetalles) y filtra si viene "buscar".
        public ActionResult Index(string buscar)
        {
            ViewBag.Buscar = buscar;
            return View(_servicio.Buscar(buscar));
        }
    }
}
```

```

// GET: Prestamos/Details/5 ? detalle del préstamo con botón "Devolver".
public ActionResult Details(int? id)
{
    if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    var prestamo = _servicio.ObtenerConDetalles(id.Value);
    if (prestamo == null) return HttpNotFound();
    return View(prestamo);
}

// GET: Prestamos/Create ? formulario para registrar un préstamo nuevo.
// Los combos de Equipo solo muestran equipos con Estado = "Disponible".
public ActionResult Create()
{
    CargarListasParaCrear();
    return View(new Prestamo { FechaPrestamo = DateTime.Now });
}

// POST: Prestamos/Create
// PrestamoServicio.RegistrarPrestamo valida que el equipo no esté ya
// prestado y, si todo está bien, marca el Equipo como "Prestado".
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Create([Bind(Include = "Id,EquipoId,EmpleadoId,FechaPrestamo,FechaEntrega")] Prestamo prestamo)
{
    if (ModelState.IsValid)
    {
        try
        {
            _servicio.RegistrarPrestamo(prestamo);
            TempData["MensajeExito"] = "Préstamo registrado correctamente.";
            return RedirectToAction("Index");
        }
        catch (InvalidOperationException ex)
        {
            ModelState.AddModelError("", ex.Message);
        }
    }

    CargarListasParaCrear(prestamo);
    return View(prestamo);
}

// GET: Prestamos/Edit/5 ? formulario de edición (Administrador y Operador).
public ActionResult Edit(int? id)
{
    if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    var prestamo = _equipoYPrestamo(id.Value);
    if (prestamo == null) return HttpNotFound();
    CargarListasParaEditar(prestamo);
    return View(prestamo);
}

[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Edit([Bind(Include = "Id,EquipoId,EmpleadoId,FechaPrestamo,FechaEntrega")] Prestamo prestamo)
{
    if (ModelState.IsValid)
    {
        // El Estatus solo cambia a través de la acción "Devolver" para mantener
        // sincronizado el estado del equipo; aquí se conserva el valor guardado.
        var actual = _servicio.ObtenerConDetalles(prestamo.Id);
        if (actual == null) return HttpNotFound();
        prestamo.Estatus = actual.Estatus;

        _servicio.Actualizar(prestamo);
        TempData["MensajeExito"] = "Préstamo actualizado correctamente.";
    }
}

```

```

        return RedirectToAction("Index");
    }

    CargarListasParaEditar(prestamo);
    return View(prestamo);
}

// POST: Prestamos/Devolver/5
// Marca el préstamo como "Devuelto" (con fecha de entrega = ahora) y
// libera el equipo (vuelve a Estado = "Disponible"). Disponible para
// Administrador y Operador: es la acción principal del día a día.
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Devolver(int id)
{
    try
    {
        _servicio.RegistrarDevolucion(id);
        TempData["MensajeExito"] = "Devolución registrada correctamente.";
    }
    catch (InvalidOperationException ex)
    {
        TempData["MensajeError"] = ex.Message;
    }

    return RedirectToAction("Details", new { id });
}

// GET: Prestamos/Delete/5 ? solo Administrador (rol extra sobre la
// clase, que ya de por sí permite Administrador y Operador).
[Authorize(Roles = "Administrador")]
public ActionResult Delete(int? id)
{
    if (id == null) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    var prestamo = _servicio.ObtenerConDetalles(id.Value);
    if (prestamo == null) return HttpNotFound();
    return View(prestamo);
}

[HttpPost, ActionName("Delete")]
[Authorize(Roles = "Administrador")]
[ValidateAntiForgeryToken]
public ActionResult DeleteConfirmed(int id)
{
    _servicio.Eliminar(id);
    TempData["MensajeExito"] = "Préstamo eliminado correctamente.";
    return RedirectToAction("Index");
}

private Prestamo _equipoYPrestamo(int id)
{
    return _servicio.ObtenerConDetalles(id);
}

private void CargarListasParaCrear(Prestamo prestamo = null)
{
    ViewBag.EquipoId = new SelectList(_equipoRepositorio.ObtenerDisponibles(), "Id", "Nombre", prestamo?.EquipoId);
    ViewBag.EmpleadoId = new SelectList(_empleadoRepositorio.ObtenerTodos(), "Id", "Nombre", prestamo?.EmpleadoId);
}

private void CargarListasParaEditar(Prestamo prestamo)
{
    // En edición se incluye también el equipo ya asignado a este préstamo,
    // aunque su estado actual sea "Prestado".
    ViewBag.EquipoId = new SelectList(_equipoRepositorio.ObtenerTodos(), "Id", "Nombre", prestamo.EquipoId);
    ViewBag.EmpleadoId = new SelectList(_empleadoRepositorio.ObtenerTodos(), "Id", "Nombre", prestamo.EmpleadoId);
}

```

```
    }

    protected override void Dispose(bool disposing)
    {
        if (disposing) db.Dispose();
        base.Dispose(disposing);
    }
}
```


6. UsuariosController

Para que sirve

Administra las CUENTAS del sistema (tabla AspNetUsers) -no confundir con Empleados, que es un catalogo de negocio y no gente que inicia sesion-. Es exclusivo de Administrador. Desde aqui se puede: listar todos los usuarios junto con sus roles actuales (Index), cambiar los roles asignados a un usuario (EditarRoles), bloquear o desbloquear su acceso sin borrar la cuenta (AlternarBloqueo), y eliminar una cuenta (excepto la propia, para no quedarse sin acceso al sistema). Todos los metodos son asincronos (async/await) porque la version de ASP.NET Identity usada en este proyecto (Microsoft.AspNet.Identity.Core 2.2.4) no ofrece versiones sincronas de estas operaciones.

Codigo completo (UsuariosController.cs)

```
using System;
using System.Linq;
using System.Net;
using System.Threading.Tasks;
using System.Web;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.ViewModels;
using Microsoft.AspNet.Identity;
using Microsoft.AspNet.Identity.Owin;
using Microsoft.Owin;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    // =====
    // UsuariosController
    // =====
    // Administración de las CUENTAS del sistema (tabla AspNetUsers), no confundir
    // con Empleados (que es un catálogo de negocio, no gente que inicia sesión).
    // Solo Administrador. Desde aquí se puede: ver todos los usuarios y sus
    // roles, cambiar los roles de un usuario, bloquear/desbloquear el acceso, y
    // eliminar una cuenta (menos la propia, para no auto-bloquearse el sistema).
    // Todos los métodos son async porque Microsoft.AspNet.Identity.Core no
    // expone versiones sincronicas de estas operaciones (FindById, Delete,
    // GetRoles, etc. solo existen como FindByIdAsync, DeleteAsync, GetRolesAsync...).
    // =====
    [Authorize(Roles = "Administrador")]
    public class UsuariosController : Controller
    {
        private ApplicationUserManager _userManager;
        private ApplicationRoleManager _roleManager;

        public ApplicationUserManager UserManager =>
            _userManager ?? (_userManager = HttpContext.GetOwinContext().GetUserManager<ApplicationUserManager>());

        public ApplicationRoleManager RoleManager =>
            _roleManager ?? (_roleManager = HttpContext.GetOwinContext().Get<ApplicationRoleManager>());

        // GET: Usuarios/Index?buscar=texto
        // Lista todos los usuarios con sus roles actuales y si están bloqueados.
        public async Task<ActionResult> Index(string buscar)
        {
            var usuarios = UserManager.Users.AsQueryable();

            if (!string.IsNullOrEmpty(buscar))
            {
                usuarios = usuarios.Where(u =>
```

```

        u.Email.Contains(buscar) ||
        u.NombreCompleto.Contains(buscar));
    }

    var lista = new System.Collections.Generic.List<UsuarioListaViewModel>();

    foreach (var usuario in usuarios.OrderBy(u => u.Email).ToList())
    {
        var roles = await UserManager.GetRolesAsync(usuario.Id);

        lista.Add(new UsuarioListaViewModel
        {
            Id = usuario.Id,
            NombreCompleto = usuario.NombreCompleto,
            Email = usuario.Email,
            Bloqueado = usuario.LockoutEndDateUtc.HasValue && usuario.LockoutEndDateUtc.Value > DateTime.UtcNow,
            Roles = roles.ToList()
        });
    }

    return View(lista);
}

// GET: Usuarios/EditarRoles/idDelUsuario
// Muestra los 2 roles fijos (Administrador/Operador) con checkboxes,
// marcando los que el usuario ya tiene asignados.
public async Task<ActionResult> EditarRoles(string id)
{
    if (string.IsNullOrEmpty(id)) return new HttpStatusCodeResult(HttpStatusCode.BadRequest);

    var usuario = await UserManager.FindByIdAsync(id);
    if (usuario == null) return HttpNotFound();

    var modelo = new EditarRolesViewModel
    {
        UsuarioId = usuario.Id,
        NombreCompleto = usuario.NombreCompleto,
        Email = usuario.Email,
        TodosLosRoles = RoleManager.Roles.Select(r => r.Name).OrderBy(r => r).ToList(),
        RolesSeleccionados = (await UserManager.GetRolesAsync(usuario.Id)).ToList()
    };

    return View(modelo);
}

// POST: Usuarios/EditarRoles
// Quita todos los roles actuales y vuelve a asignar solo los que
// llegaron marcados desde el formulario (rolesSeleccionados).
// IMPORTANTE: el rol se "congela" en la cookie de sesión al iniciar
// sesión, así que el usuario debe volver a iniciar sesión para que
// el cambio de rol le tome efecto.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> EditarRoles(string usuarioId, System.Collections.Generic.List<string> rolesSeleccionados)
{
    var usuario = await UserManager.FindByIdAsync(usuarioId);
    if (usuario == null) return HttpNotFound();

    var rolesActuales = await UserManager.GetRolesAsync(usuarioId);
    rolesSeleccionados = rolesSeleccionados ?? new System.Collections.Generic.List<string>();

    if (rolesActuales.Any())
    {
        await UserManager.RemoveFromRolesAsync(usuarioId, rolesActuales.ToArray());
    }
}

```

```

        if (rolesSeleccionados.Any())
        {
            await UserManager.AddToRolesAsync(usuarioId, rolesSeleccionados.ToArray());
        }

        TempData["MensajeExito"] = "Roles actualizados correctamente.";
        return RedirectToAction("Index");
    }

    // POST: Usuarios/AlternarBloqueo/idDelUsuario
    // Bloquea o desbloquea el acceso del usuario poniendo (o quitando) una
    // fecha de bloqueo muy lejana en LockoutEndDateUtc. Un usuario bloqueado
    // no puede iniciar sesión aunque su contraseña sea correcta.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> AlternarBloqueo(string id)
    {
        var usuario = await UserManager.FindByIdAsync(id);
        if (usuario == null) return HttpNotFound();

        var bloqueadoActualmente = usuario.LockoutEndDateUtc.HasValue && usuario.LockoutEndDateUtc.Value > DateTime.UtcNow;

        if (bloqueadoActualmente)
        {
            await UserManager.SetLockoutEndDateAsync(id, DateTimeOffset.UtcNow);
            TempData["MensajeExito"] = "Usuario desbloqueado.";
        }
        else
        {
            await UserManager.SetLockoutEndDateAsync(id, DateTimeOffset.UtcNow.AddYears(100));
            TempData["MensajeExito"] = "Usuario bloqueado.";
        }

        return RedirectToAction("Index");
    }

    // POST: Usuarios/Delete/idDelUsuario
    // Elimina la cuenta, salvo que sea la cuenta con la que se está
    // conectado en este momento (para no quedarse sin acceso al sistema).
    [HttpPost, ActionName("Delete")]
    [ValidateAntiForgeryToken]
    public async Task<ActionResult> DeleteConfirmed(string id)
    {
        var usuario = await UserManager.FindByIdAsync(id);
        if (usuario == null) return HttpNotFound();

        if (usuario.UserName == User.Identity.Name)
        {
            TempData["MensajeError"] = "No puedes eliminar tu propia cuenta.";
            return RedirectToAction("Index");
        }

        await UserManager.DeleteAsync(usuario);
        TempData["MensajeExito"] = "Usuario eliminado correctamente.";
        return RedirectToAction("Index");
    }

    protected override void Dispose(bool disposing)
    {
        if (disposing)
        {
            _userManager?.Dispose();
            _roleManager?.Dispose();
        }
        base.Dispose(disposing);
    }

```

```
}  
}
```

7. RolesController

Para que sirve

Es una pantalla de solo lectura para consultar los dos roles fijos del sistema (Administrador y Operador) y cuantos usuarios tiene asignado cada uno. A proposito NO permite crear ni eliminar roles desde la interfaz: el sistema esta disenado para trabajar unicamente con estos dos roles, sembrados por el script Script_TechSolutionsDB.sql. Para cambiar el rol de un usuario se usa la pantalla Usuarios -> Roles, no esta.

Codigo completo (RolesController.cs)

```
using System.Linq;
using System.Threading.Tasks;
using System.Web;
using System.Web.Mvc;
using Caso.Uno.Principal.Front.views.Datos;
using Caso.Uno.Principal.Front.views.ViewModels;
using Microsoft.AspNet.Identity.Owin;
using Microsoft.Owin;

namespace Caso.Uno.Principal.Front.views.Controllers
{
    /// <summary>
    /// El sistema trabaja con exactamente dos roles fijos (Administrador y
    /// Operador), creados por Database/Script_TechSolutionsDB.sql. Esta
    /// pantalla es de solo lectura: no se permite crear ni eliminar roles,
    /// solo consultar cuántos usuarios tiene cada uno.
    /// </summary>
    [Authorize(Roles = "Administrador")]
    public class RolesController : Controller
    {
        private ApplicationRoleManager _roleManager;
        private ApplicationUserManager _userManager;

        public ApplicationRoleManager RoleManager =>
            _roleManager ?? (_roleManager = HttpContext.GetOwinContext().Get<ApplicationRoleManager>());

        public ApplicationUserManager UserManager =>
            _userManager ?? (_userManager = HttpContext.GetOwinContext().Get<ApplicationUserManager>());

        // GET: Roles/Index
        // Para cada uno de los 2 roles, cuenta cuántos usuarios lo tienen
        // asignado (recorriendo todos los usuarios y preguntando IsInRoleAsync).
        // Es de solo lectura: no hay Create/Delete de roles en este controlador.
        public async Task<ActionResult> Index()
        {
            var usuarios = UserManager.Users.ToList();
            var roles = new System.Collections.Generic.List<RolListaViewModel>();

            foreach (var rol in RoleManager.Roles.OrderBy(r => r.Name).ToList())
            {
                var total = 0;
                foreach (var usuario in usuarios)
                {
                    if (await UserManager.IsInRoleAsync(usuario.Id, rol.Name))
                    {
                        total++;
                    }
                }

                roles.Add(new RolListaViewModel { Id = rol.Id, Name = rol.Name, TotalUsuarios = total });
            }
        }
    }
}
```

```
        return View(roles);
    }

    protected override void Dispose(bool disposing)
    {
        if (disposing)
        {
            _roleManager?.Dispose();
            _userManager?.Dispose();
        }
        base.Dispose(disposing);
    }
}
```